ECE 553/653 Neural Networks

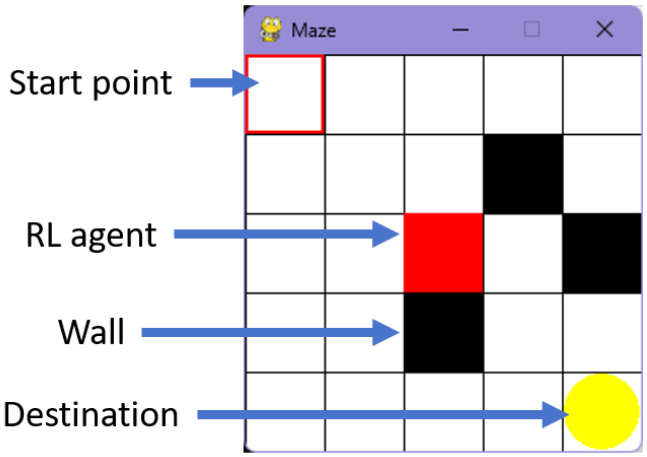
Project 2: Q-Learning and Deep Q-Network

In this project, you will implement Q-Learning and Deep Q-Network (DQN) algorithms to optimize the behavior of a reinforcement learning (RL) agent applied to a maze pathfinding problem. Your task includes designing and tuning these algorithms to enable effective agent navigation.

1. Requirements

You will complete **two** programs: a program that optimizes the RL agent's behaviors by Q-Learning, and a program that optimize RL agent's behaviors by DQN. It is recommended that your programs are written in Python. For additional Python environment install, please consult the "README.md" file.

2. Program Introduction



The Scenario.

The program comprises the following components:

1. **"main.py"**: Initializes the experimental environment and learning parameters, and executes the main training loop.
2. **"QLearning.py"**: Implements the Q-Learning algorithm through the "QTable" class.
3. **"DQN.py"**: Contains the DQN implementation via the "QNetwork" class.
4. **"utils.py"**: Provides environment definitions (implemented in the MAZE class) and visualization utilities for reward plotting.

3. Task Introduction

Part I: Q-Learning tasks

1. **Action Selection Mechanism**: Examine the "QTable.step()" function in "QLearning.py" to implement an action selection strategy that incorporates the exploration probability (self.epsilon) and the current Q-table (self.q_table). For implementation guidance, you may refer to the analogous action selection mechanism in "QNetwork.step()" function within DQN.py.
2. **Qtable Updates Algorithm Design**: Review the "QTable.update_q_table()" function in

"QLearning.py" to implement the Q-table update rule based on the (state, action, next_state) triplet using the standard Q-learning formulation.

Part II: DQN tasks

3. **DQN Architecture Design:** Inspect the "QNetwork.__init__()" function in DQN.py to construct a fully connected neural network (QNetwork.network) that maps state inputs to action outputs in compliance with PyTorch conventions.
4. **Expected Q-Value Calculation:** Analyze the "QNetwork.learn()" function in DQN.py to compute the expected Q-values using the (state, action, next_state, reward) tuple and the target network.

Part III: Overall task

5. **Hyperparameter Tuning and Evaluation**: Test different hyperparameter configurations in main.py, and compare their impact on learning performance and computational efficiency.

4. Report

You will write a report for this project. There is no strict format of this report but below are the essential parts:

1. Action Selection Mechanism: Describe the design and implementation of your action selection strategy, explaining how it balances exploration and exploitation.
2. Algorithm Design: Explain the update rules you implemented for both the Q-table and the Q-Network. Additionally, discuss the key differences between these two update mechanisms.
3. Experimental Results: Include a screenshot showing the successful performance of your trained agent in the environment..
4. Hyperparameter Analysis: Experiment with different training configurations (e.g., learning rate, discount factor, exploration rate) and report on their impact on the learning process and outcomes.

* Do not paste a lot of codes in the report. Focus on the design and analysis. Your codes will be submitted separately.

5. Submitting your project

You should submit your project in Blackboard. The submission includes: a project report and your source codes. Please put everything in a zip/rar file, name it “YourFirstName_YourLastName_Project3.zip”.

6. Advice

Start early! Do not wait until the last few days.